BFOR516 - Representing Data and Engineering Features

Srishti Gupta, Ph.D.

Feature Engineering

Introduction

- Most of the data we encounter in machine learning examples is continuous data.
  - Data is in the form of a table with numbers, where each column represents a continuous measurement, like the height of a person or the brightness of a pixel in an image.
  - They can take any value within a range.
- However, in many real-world applications, data isn't always numerical like this. Instead, some features are categorical, meaning they belong to distinct groups rather than a continuous range. These are also called discrete features.
  - A product's brand (Nike, Adidas, Puma)
  - The color of a product (Red, Blue, Green)
  - The department a product is sold in (Books, Clothing, Hardware)

Importance of Data Representation

- The way you represent your data can greatly impact how well a machine-learning model performs. For instance:
  - If you don't scale numerical data properly, a model may give too much importance to one feature over another (e.g., if one is measured in inches and another in centimeters).
  - Sometimes, combining or transforming features can improve performance (e.g., multiplying two features or using polynomial transformations).
- The process of selecting and transforming features in a way that makes the model work better is called feature engineering.

Categorical Variables

- Categorical variables are features in a dataset that represent categories rather than numerical values. Unlike numbers, these categories do not have a meaningful mathematical order.
- Example from the Adult Income Dataset:
  - Imagine a dataset where we predict whether a person earns more or less than $50,000 per year based on features like:
    - Age (numerical)
    - Workclass (category: government, private, self-employed, etc.)
    - Education (category: high school, bachelor's, master's, etc.)
    - Gender (category: male, female)
    - Hours worked per week (numerical)
    - Occupation (category: executive, clerical, service, etc.)
  - Some features (like age and hours worked) are numbers, which can be directly used in a mathematical formula. However, workclass, education, gender, and occupation are categories (words) and cannot be used directly in a mathematical model.

Why Do We Need to Convert Categorical Variables?

- Most machine learning models (like logistic regression) work with numbers, not text.
- For example, a logistic regression model makes predictions using this formula:

$$\hat{y} = w[0] * x[0] + w[1] * x[1] + \ldots + w[p] * x[p] + b > 0$$

- Here, x represents input features, but if x is a word (like "Masters" or "Bachelors"), the equation doesn't work. We need to convert categorical variables into a numerical format.

One-Hot Encoding

- One way to convert categorical variables into numbers is One-Hot Encoding. This method turns each category into a new column with values 0 or 1.
- Example: Encoding the Workclass Feature
  - Let's say we have four possible values for workclass:
    - Government Employee
    - Private Employee
    - Self-Employed
    - Self-Employed Incorporated

One-Hot Encoding (Example)

Instead of using these labels directly, we create four new columns, one for each category. A value of 1 means that a person belongs to that category, while 0 means they don't. So, if a person works in the private sector, their encoded row would look like: [0, 1, 0, 0]. This method ensures that we represent categorical data in a way that a model can understand.

Original Workclass	**Government Employee**	**Private Employee**	**Self-Employed**	**Self-Employed Incorporated**
Government Employee	1	0	0	0
Private Employee	0	1	0	0
Self-Employed	0	0	1	0
Self-Employed Inc.	0	0	0	1

Key Benefits of One-Hot Encoding

- Converts text categories into numerical data
- Prevents the model from incorrectly assuming an order in categorical values
- Works well with most machine learning models

One-Hot Encoding in Python

Instead of manually creating new columns, we can use Pandas, a popular Python library, to automatically perform one-hot encoding.

```python
import pandas as pd

# Load dataset
data = pd.read_csv("adult.data", header=None, index_col=False,
                   names=['age', 'workclass', 'education', 'gender',
                          'hours-per-week', 'occupation', 'income'])

# Select relevant columns
data = data[['age', 'workclass', 'education', 'gender', 'hours-per-week',
             'occupation', 'income']]

# Apply one-hot encoding
data_encoded = pd.get_dummies(data, columns=['workclass', 'education', 'gender', 'occupation'])

# Display first 5 rows
print(data_encoded.head())
```

Checking and Encoding Categorical Data

When working with datasets that have categorical data (like gender or workclass), it's important to check how this data is stored before using it in a machine learning model.

- Sometimes, people enter the same category in different ways (e.g., "male" vs. "man").
- There may be inconsistent capitalizations ("Male" vs. "male").
- Some categories might have extra spaces or typos.

To check how the categorical column looks, we use:

```
data.gender.value_counts()
```

This shows all unique values in the gender column and how many times each appears.

Encoding Categorical Data with `get_dummies()`

Before Encoding:

```
['age', 'workclass', 'education', 'gender', 'hours-per-week',
'occupation', 'income']
```

- `age` and `hours-per-week` are numerical → unchanged
- `workclass, education, gender, occupation,` and `income` are categorical → need encoding

After Applying `get_dummies()`:

```
data_dummies = pd.get_dummies(data)
print(list(data_dummies.columns))
```

```
['age', 'hours-per-week', 'workclass_ Private', 'workclass_
Government','education_ Bachelors', 'education_ Masters', 'gender_
Female', 'gender_ Male','occupation_ Tech-support', 'income_ <=50K',
'income_ >50K']
```

Separate input (X) and output (y)

```
X = data_dummies.drop(columns=['income_<=50K',
'income_>50K']).values
y = data_dummies[['income_<=50K',
'income_>50K']].values
```

X contains input features.

y contains target labels.

Binning (Discretization)

- The best way to represent data depends not only on the semantics of the data, but also on the kind of model you are using.
- Linear Models vs. Tree-Based Models:
  - Linear models (like Linear Regression) assume a straight-line relationship between input and output. They struggle with complex patterns.
  - Tree-based models (like Decision Trees, Random Forests) split the data into different sections based on conditions, making them flexible for non-linear relationships.

Binning (Discretization)

- Binning helps linear models because it allows them to assign different values to different sections of data, making them more flexible.
  - Imagine you have a continuous range of numbers, like temperatures from -3°C to 3°C. Instead of treating them as exact values, you divide this range into smaller sections or "bins."
  - For example, if you create 10 bins, temperatures between -3°C to -2.4°C go into one bin, -2.4°C to -1.8°C in another, and so on.
  - This process turns a continuous variable into a categorical one (each data point is now assigned to a bin instead of being a precise number).

Effect of Binning on Models

For Linear Models:

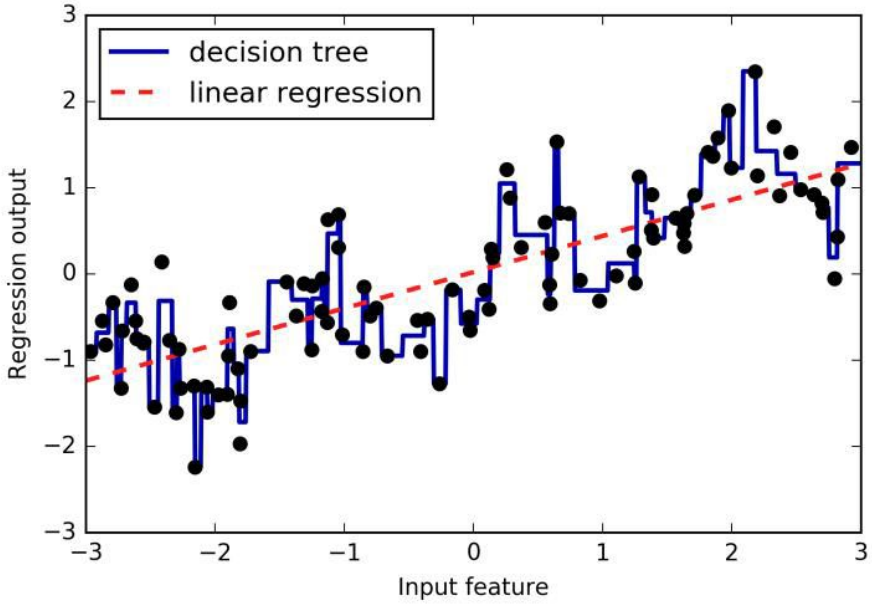
- Binning helps linear models because it allows them to assign different values to different sections of data, making them more flexible.
- Without binning, a linear model can only draw a straight line, missing patterns in the data.

For Tree-Based Model:

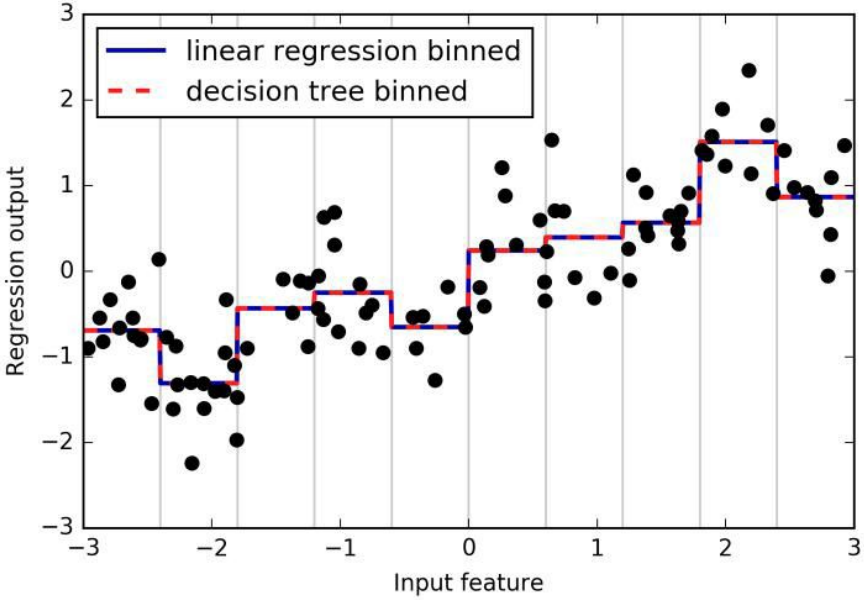
- Binning is not useful because trees naturally split the data where needed.
- Trees can learn better without bins because they make their own decisions on where to divide the data.

Effect of Binning on Models

Before Binning



After Binning



Interactions and Polynomial Features

Sometimes the relationships in the data are more complex than a straight line. To help our model capture these complexities, we can modify our features using two key techniques:

- Adding Interaction Features
- Using Polynomial Features

Interaction Features (Combining Features for Better Learning)

- **Issue with binning in linear models**: A simple binning approach doesn't allow for different trends within each bin.
- To improve this, we add **interaction features**, which **combine the original feature (X) with its bin indicators**. This allows the model to learn a different slope for each bin instead of just a fixed value per bin. Example: Instead of saying,
  - "For bin 1, output is always 2,"
  - "For bin 2, output is always 5,"
  - We can now say:
  - "For bin 1, output changes based on X,"
  - "For bin 2, output also changes based on X."
- This gives the model more flexibility to learn different trends in each region of the data.

Polynomial Features (Adding Powers of Features for Non-Linearity)

- **Issue with Linear Models**: Might not be enough to capture curves in data.
- Instead of just using X, we can create **polynomial features like: X^2, X^3**, and so on
- This allows our model to learn curved patterns in the data, making it more powerful.
- Example:
  - If the real relationship between X and Y looks like a U-shape or wave, a straight line can't capture that well.
  - But if we add X^2 and X^3, the model can now fit curves!
- By using `PolynomialFeatures` from `sklearn`, we automatically generate these new features.

When to Use Interaction and Polynomial Features

The choice depends on the type of model and data characteristics.

When to Use Interaction Features:

- When using linear models (like Linear Regression, Ridge, Lasso), which cannot capture feature dependencies by default.
- When you suspect feature interactions (e.g., in medical studies, the effect of a drug may depend on both dosage and age).
- When interpretability is important (as interaction terms are easier to explain than higher-degree polynomials).

When to Use Interaction and Polynomial Features

When to Use Polynomial Features:

- When you expect non-linear relationships but still want to use a linear model.
- For algorithms that don't automatically learn non-linearity, such as linear regression, logistic regression, or support vector machines.
- When using low-degree polynomials (e.g., degree=2 or 3) to avoid overfitting.

When to Use Interaction and Polynomial Features

When Not to Use Polynomial or Interaction Features:

- With tree-based models (Random Forest, XGBoost, Decision Trees): These models naturally capture non-linearity and interactions.
- With high-dimensional data: Adding polynomial features significantly increases feature count, leading to overfitting.
- With deep learning models: Neural networks learn complex patterns automatically, making explicit polynomial features unnecessary.

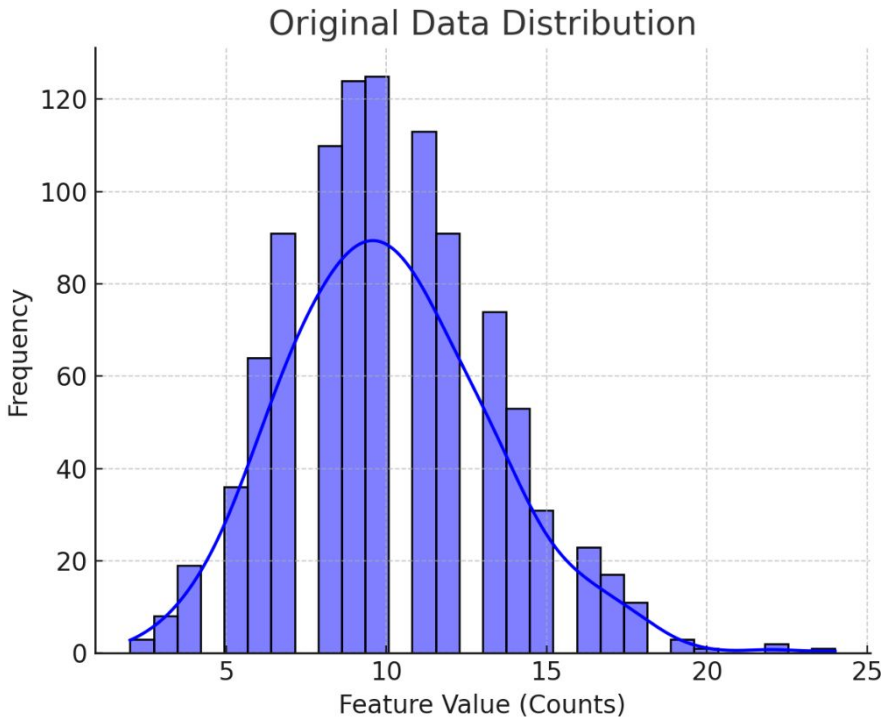
Univariate Nonlinear Transformations

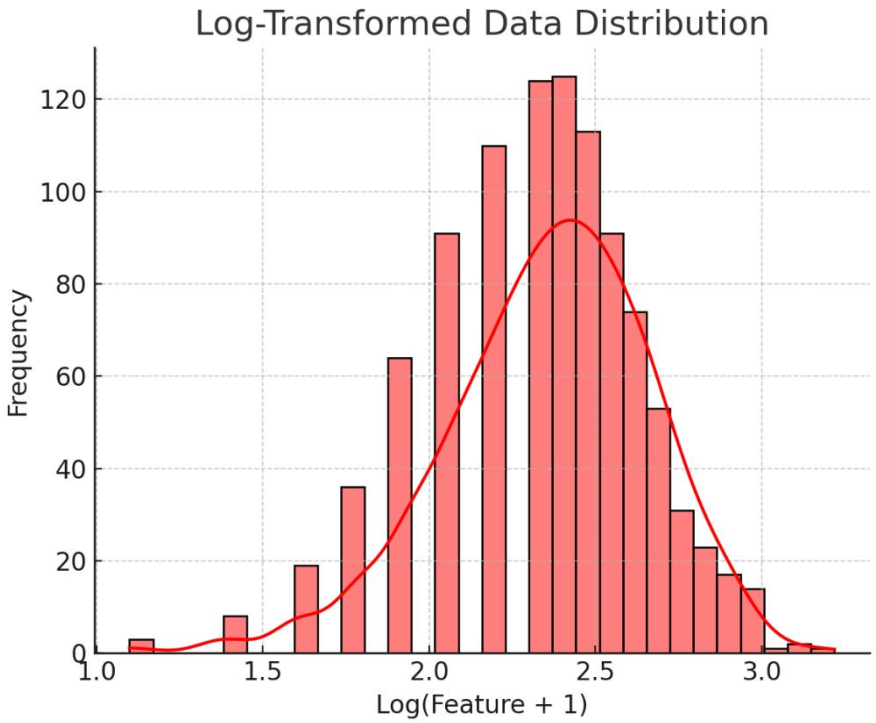
- When using machine learning models, especially linear models and neural networks, the way the data is distributed matters a lot.
- If a feature has a very skewed or nonlinear relationship with the target (non-Gaussian), the model may struggle to learn from it.
- To address this, we can apply univariate nonlinear transformations—mathematical functions like `log`, `exp`, and `sin`—to reshape the data and make it easier for the model to learn.

Common Transformations and When to Use Them

Transformation	Formula	When to Use?
Logarithm (log)	log(x + 1)	When the feature has a long tail (many small values, few large values)
Exponential (exp)	exp(x)	When the relationship between the feature and target grows exponentially
Square root (sqrt)	sqrt(x)	When the differences between smaller values matter more than larger ones
Sine & Cosine (sin, cos)	sin(x), cos(x)	When dealing with cyclical data (e.g., time of day, seasons)

Common Transformations and When to Use Them





Feature Selection

Feature Selection

- Adding too many features can make your model overly complex and prone to overfitting (where the model performs well on training data but poorly on new, unseen data).
- To avoid this, it's a good idea to keep only the most useful features and discard the rest. This process is called feature selection, and it helps create simpler models that perform better on new data.

3 Main Ways to do Feature Selection

1. Univariate Statistics: Check each feature individually to see how strongly it relates to the target.
2. Model-Based Selection: Use a machine learning model to determine which features are most important.
3. Iterative Selection: Try different combinations of features to find the best set.

All these methods are supervised, meaning they need to know the target variable to work. You also need to split your data into a training set (to select features) and a test set (to evaluate the model).

Univariate Statistics

It looks at each feature one by one and checks if it has a meaningful relationship with the target. For example:

- In classification (predicting categories), it uses ANOVA.
- In regression (predicting numbers), it uses F-test.

The key idea is to keep only the features that are most strongly related to the target.

Model-Based Feature Selection

Instead of looking at each feature individually (like in univariate statistics), model-based feature selection uses a machine learning model to determine which features are most important.

The idea is to train a model and use it to rank the features based on their importance. The model used for feature selection doesn't have to be the same as the final model you'll use for predictions.

How It Works:

- Train a model (like a decision tree, random forest, or linear model) on your data.
- The model provides a measure of feature importance:
- Decision trees and random forests have a feature_importances_ attribute.
- Linear models use the absolute values of their coefficients to determine importance.
- Keep only the features with the highest importance scores.

Iterative Feature Selection

This is a more advanced method where you build multiple models to find the best set of features. It's slower but can be more accurate because it tries different combinations of features.

How It Works: There are two main approaches:

- **Forward Selection**: Start with no features and add one feature at a time until the model performance stops improving.
- **Backward Elimination (Recursive Feature Elimination, RFE):** Start with all features and remove the least important feature one at a time until only a certain number of features remain.
  - Train a model (like a random forest) on all features.
  - Remove the least important feature.
  - Repeat the process until only the desired number of features is left.

Textbook Reference

Müller, A. C., & Guido, S. (2016). Introduction to machine learning with Python: a guide for data scientists. " O'Reilly Media, Inc.". - Chapter 4